Backend Connection Guide (CityPulse Flutter)

This document explains **exactly where** to connect your backend API in this Flutter project and what to implement so the app uses real data instead of mock data.

1) The “one place” for base URL + auth headers

File: `lib/core/services/api_service.dart`

What it does

• Stores and reads the JWT access token from secure storage.

• Automatically attaches the header `Authorization: Bearer ` to requests.

• Provides helper methods for:

- JSON `GET`, `POST`, `PATCH`

- `Multipart POST` for uploading images + fields

What you change

• Update `kBaseUrl` to your backend base URL:

- Example (LAN): `http://192.168.1.10:8000/api`

- Android emulator: use `http://10.0.2.2:8000/api` (NOT `localhost`)

2) Where ALL issue/complaint backend integration happens

File: `lib/features/issues/repositories/remote_issue_repository.dart`

This is the **main integration point**. Screens and providers already call this repository, so when you implement real API calls here, the UI starts using your backend automatically.

Replace mock code in these methods

• `classifyImage(...)`

• `submitComplaint(...)`

• `fetchComplaints(...)`

• `fetchMyComplaints()`

• `upvoteComplaint(complaintId)`

• `fetchDashboard()`

• `fetchNotifications()`

• `markNotificationsRead()`

JSON mapping

Function: `_issueFromJson(Map<String, dynamic> json)`

Update this to match your backend response keys for:

• `id`, `complaint_number`, `issue_type`

• `description`, `status`

• `latitude`, `longitude`, `address`
• `user` (owner), image URL fields (e.g. `primary_image`)

3) Where AUTH (login/register) backend integration happens

File: `lib/features/auth/controllers/auth_controller.dart`

Right now `login()` and `register()` are **mock** (they only save local session info).

What to implement
• Call your backend login/register endpoint.
• When backend returns a JWT access token, store it:
- `ApiService.saveToken(token)`
• Keep caching user data in `SharedPreferences` if you want “fast session restore”.

Should frontend send user name to backend?
Recommended: No. The backend should identify the user from the JWT (`request.user`).

If you want to display the user name in the UI, the backend should **return it** in API responses (or provide a profile endpoint).

4) Provider wiring (already done)

File: `lib/features/issues/providers/issue_providers.dart`

• `remoteIssueRepositoryProvider` is the repository used throughout the app.
• No need to change screens when you replace mocks with real API calls.

5) Practical integration order (safe path)

1. Set `kBaseUrl` correctly in `ApiService`.
2. Implement `AuthController.login()` → save JWT via `ApiService.saveToken`.
3. Implement `RemoteIssueRepository.fetchComplaints()` and `_issueFromJson`.
4. Implement `RemoteIssueRepository.submitComplaint()` using `ApiService.postMultipart`.
5. Implement “My complaints” endpoint in `fetchMyComplaints()`.

6) Local testing notes

• **Web** can use LAN IP or localhost (depending on backend).
• **Android emulator**: use `10.0.2.2` to reach your PC.
• **Physical Android device**: use your PC LAN IP, ensure firewall allows inbound traffic.